

is there an ai that will categorize videos based on its title?

Yes, there are AI tools and models that can categorize videos based **only on their titles**, though the accuracy may be limited compared to analyzing the video content itself. These systems typically use **natural language processing (NLP)** to infer the topic or category from the text in the title.

Options You Can Use:

1. Custom AI Models (Using OpenAI, Hugging Face, etc.)

You can create a simple script using a **pre-trained text classification model** (like BERT or GPT) to classify video titles into categories such as:

- Education
- Entertainment
- News
- Sports
- Technology
- Gaming, etc.

Example tools:

- **Hugging Face Transformers** (with models like `distilbert-base-uncased`)
- **OpenAI GPT API** (you can prompt it with something like: *"Categorize this video title into one of [Education, Sports, Gaming...]: 'Top 10 Python Tricks'"*)

2. YouTube Data API + Custom Classifier

If you're pulling video titles from YouTube:

- Use the **YouTube Data API** to get titles and metadata.
- Apply a classifier (using NLP libraries like **spaCy**, **scikit-learn**, or **transformers**) to categorize them.

3. Third-party AI Tools

There are platforms that offer AI tagging and categorization:

- **Clarifai or Hive AI:** These usually work better when analyzing full video content, but some support text-only metadata input.
- **Google Cloud Video Intelligence:** Mostly content-based, but can be paired with NLP.

4. Zapier + ChatGPT Plugin or API

You can automate title categorization by:

- Setting up a Zapier workflow to send new video titles to ChatGPT or another NLP service.
- Have ChatGPT return a category based on a predefined list.

Sample Prompt (if using ChatGPT API or GPT manually):

Given the following video title: "Best Budget Laptops for Students in 2025"
Categorize it **as** one **of**: [Technology, Education, Entertainment, Sports, News]

Answer:

Would you like help building a small script that does this? If you provide a few titles, I can also show you how well it works in practice.

what i have are youtube videos from extra history. they do short series on a topic (ie can be 2 or typically 3-6 videos). i want to classify these like episodes together only through their title. but i also have their published dates to help with if it matters, since these serial videos are released closely to each other, almost always 1 week since the previous episode

Great — that's a clear and doable task. Since you're working with **Extra History** videos (which tend to follow naming conventions like "Julius Caesar - I: The Fall of the Republic" , "Julius Caesar - II: Crossing the Rubicon" , etc.), you can group them into series using **patterns in their titles**, and optionally use **publish dates** to resolve ambiguities or validate sequence.



Your Goal

Group Extra History videos into **series** based on title patterns and possibly date proximity — even without access to the full video content.



Strategy (Only Using Title and Publish Date)

1. Title Pattern Matching

Most Extra History series use a naming structure like:

<Series **Name**> - <Episode **Number** or **Roman Numeral**>: <Subtitle>

Examples:

- Julius Caesar - I: The Fall of the Republic
- Julius Caesar - II: Crossing the Rubicon

From this, we can extract:

- Series Name : everything before the hyphen
- Episode Index : Roman numeral or number
- Subtitle : not necessary for grouping

2. Use Publish Date for Extra Confidence

- If two titles are from the same series (same Series Name) and are **within 7–10 days** of each other, they're almost certainly part of the same run.

- This helps avoid false positives in rare cases where two series might share similar names.

Implementation Plan

You can do this with Python. Here's a rough sketch of the logic:

Input:

- A list of video objects with:
 - title
 - publish_date

Output:

- Grouped series (list of lists or dict of series → list of episodes)

Sample Code:

```
import re from datetime import datetime, timedelta from collections import
defaultdict def extract_series_key(title): match = re.match(r"(.+?)\s*-\s*
[IVXLCDM0-9]+:", title) if match: return match.group(1).strip() return None
def group_by_series(videos): # videos = list of dicts with 'title' and
'published' (as datetime) series_dict = defaultdict(list) for video in videos:
key = extract_series_key(video['title']) if key:
series_dict[key].append(video) # Optionally sort episodes by publish date
within each series for key in series_dict: series_dict[key] =
sorted(series_dict[key], key=lambda x: x['published']) return
dict(series_dict)
```

Example Input

```
videos = [ {"title": "Julius Caesar - I: The Fall of the Republic",
"published": datetime(2022, 5, 1)}, {"title": "Julius Caesar - II: Crossing
the Rubicon", "published": datetime(2022, 5, 8)}, {"title": "WWI - I: The Road
```



```
to War", "published": datetime(2022, 6, 1)}, {"title": "WWI - II: The Guns of  
August", "published": datetime(2022, 6, 8)}, ]
```

Output

```
{ "Julius Caesar": [ {"title": "...", "published": ...}, {"title": "...",  
"published": ...} ], "WWI": [ ... ] }
```

Want Me to Help Further?

- You can upload a CSV/JSON of your video titles and dates.
- Or I can give you a full working script you can run locally or in Google Colab.

Let me know how hands-on you want to get with it!

no, use something like hdbscan

Got it — you want to cluster the **titles into series** using **HDBSCAN**, which is a density-based clustering algorithm.

That's a smart approach — it allows for **flexible, unsupervised grouping** based on **semantic similarity**, and it doesn't require knowing the number of clusters in advance (unlike K-means).

 **Updated Plan: Using HDBSCAN to Group Video Titles into Series**

Tools

- **Sentence Transformers (e.g., all-MiniLM-L6-v2):** To convert titles into vector embeddings.
- **HDBSCAN:** To cluster similar titles.
- **Optionally:** Use publish dates to post-process or refine clusters.

Workflow

1. **Convert titles to embeddings** using a sentence transformer.
2. **Cluster embeddings** with HDBSCAN.
3. **Assign videos to clusters** (each representing a series).
4. **(Optional)** Sort cluster members by publish date.

Example Code

Here's a complete working pipeline (Python):

```
import hdbscan from sentence_transformers import SentenceTransformer import
pandas as pd from datetime import datetime # Step 1: Your input data videos =
[ {"title": "Julius Caesar - I: The Fall of the Republic", "published": "2022-
05-01"}, {"title": "Julius Caesar - II: Crossing the Rubicon", "published":
"2022-05-08"}, {"title": "WWI - I: The Road to War", "published": "2022-06-
01"}, {"title": "WWI - II: The Guns of August", "published": "2022-06-08"},
{"title": "Hannibal - I: Rome's Greatest Enemy", "published": "2022-07-01"},
{"title": "Random One-Off History", "published": "2022-04-01"}, ] # Convert to
DataFrame df = pd.DataFrame(videos) df["published"] =
pd.to_datetime(df["published"]) # Step 2: Embed titles model =
SentenceTransformer('all-MiniLM-L6-v2') embeddings =
model.encode(df["title"].tolist()) # Step 3: Cluster with HDBSCAN clusterer =
hdbscan.HDBSCAN(min_cluster_size=2, metric='euclidean') labels =
clusterer.fit_predict(embeddings) df["series_cluster"] = labels # Step 4: Sort
within clusters by date df_sorted = df.sort_values(by=["series_cluster",
"published"]) # Display clusters for cluster_id in
sorted(df["series_cluster"].unique()): print(f"\nCluster {cluster_id} if
```

```
cluster_id != -1 else 'Unclustered'}:")
print(df_sorted[df_sorted["series_cluster"] == cluster_id][["title",
"published"]])
```

Notes

- `-1` in `series_cluster` means HDBSCAN marked it as noise (i.e. unclustered).
- You can adjust `min_cluster_size` and other HDBSCAN parameters for tighter or looser grouping.
- You can also integrate **date difference filtering** post-clustering to merge/split borderline cases.

Requirements

Install the necessary packages if you haven't:

```
pip install hdbscan sentence-transformers pandas
```

Optional Enhancements

- Filter out or re-cluster `-1` (noise) items separately.
- Incorporate **publish date** into a **custom distance function**, though HDBSCAN doesn't support that natively — you'd need to do hybrid embedding (text + date vector) and tune weights.

Would you like a Colab version of this to try right away? Or want to upload your data for testing?

this is the format of my csv data

```
Rome: The Punic Wars - The Conclusion of the Second  
Punic War - Extra History - Part 4,2013-09-  
27,2326349,41229,2174,McT1H-  
NVCMQ,700.0,https://www.youtube.com/watch?v=McT1H-  
NVCMQ  
Rome: The Punic Wars - The Second Punic War Rages On -  
Extra History - Part 3,2013-09-  
20,2527809,35544,1798,wT_rev5VAQc,758.0,https://www.y  
outube.com/watch?v=wT_rev5VAQc  
Rome: The Punic Wars - The Second Punic War Begins -  
Extra History - Part 2,2013-09-13,2965306,38130,1510,lf0-  
Yki5p40,666.0,https://www.youtube.com/watch?v=lf0-  
Yki5p40  
Rome: The Punic Wars - The First Punic War - Extra History  
- Part 1,2013-09-  
06,5128436,67036,3038,EbBHK_zLTmY,553.0,https://www.y  
outube.com/watch?v=EbBHK_zLTmY
```

Perfect — thanks for sharing your data format.

Let's build a working solution that:

1. **Loads your CSV.**
2. **Embeds the video titles** using a sentence transformer.
3. **Clusters them** with HDBSCAN based on semantic similarity.
4. **Outputs grouped series**, optionally sorted by publish date.



Assumptions Based on Your CSV Format

Your columns are (in order):

```
title,published,date,views,likes,comments,video_id,duration,url
```

We only need:

- title (for clustering)
- published (for ordering episodes)

Full Python Script

```
import pandas as pd import hdbscan from sentence_transformers import
SentenceTransformer from datetime import datetime # === Step 1: Load the CSV
=== # Replace this with the path to your actual CSV file df =
pd.read_csv("extra_history_videos.csv", header=None) # Assign column names
df.columns = [ "title", "published", "views", "likes", "comments", "video_id",
"duration", "url" ] # Convert publish date to datetime df["published"] =
pd.to_datetime(df["published"]) # === Step 2: Embed the Titles === model =
SentenceTransformer("all-MiniLM-L6-v2") embeddings =
model.encode(df["title"].tolist()) # === Step 3: Cluster with HDBSCAN ===
clusterer = hdbscan.HDBSCAN(min_cluster_size=2, metric="euclidean")
df["series_cluster"] = clusterer.fit_predict(embeddings) # === Step 4: Sort
each cluster by publish date === df_sorted = df.sort_values(by=
["series_cluster", "published"]) # === Step 5: Output Clustered Series === for
cluster_id in sorted(df_sorted["series_cluster"].unique()): print(f"\n===
Cluster {cluster_id} if cluster_id != -1 else 'Unclassified' ===") cluster_df =
df_sorted[df_sorted["series_cluster"] == cluster_id] for _, row in
cluster_df.iterrows(): print(f"{row['published'].date()} - {row['title']}")
```



Installation Requirements

If you haven't installed the needed packages:

```
pip install pandas hdbscan sentence-transformers
```

Example Output

For your example dataset, you'll likely get one clean cluster like this:

```
=== Cluster 0 ===
2013-09-06 - Rome: The Punic Wars - The First Punic War - Extra History - Part 1
2013-09-13 - Rome: The Punic Wars - The Second Punic War Begins - Extra History -
2013-09-20 - Rome: The Punic Wars - The Second Punic War Rages On - Extra History
2013-09-27 - Rome: The Punic Wars - The Conclusion of the Second Punic War - Extr
```

Optional Enhancements

If you have many videos, you can:

- Export the cluster results to a new CSV:

```
df_sorted.to_csv("clustered_extra_history.csv", index=False)
```

- Adjust HDBSCAN params (`min_cluster_size` , `min_samples`) for different grouping sensitivity.

Would you like me to help you turn this into a Colab notebook or support date-based clustering refinement (e.g., merging small clusters within 10 days)?

what are some alternatives to all-MiniLM-L6-v2

Great question — if you're using `all-MiniLM-L6-v2` from `SentenceTransformers`, there are several strong alternatives depending on your needs like **accuracy**, **speed**, **multilingual support**, or **model size**.

Here's a quick overview:



Best Alternatives to all-MiniLM-L6-v2

1. all-MiniLM-L12-v2

- 🔍 Slightly more accurate than L6-v2 (deeper transformer: 12 layers vs. 6)
- ⚡ A bit slower and heavier
- ✅ Best drop-in replacement if you want a balance of speed and better clustering

2. paraphrase-MiniLM-L6-v2

- 🧠 Trained for paraphrase mining and semantic similarity
- ✅ Good for grouping similar titles
- 📁 Slightly older than all-MiniLM-* family, but still solid

3. all-mpnet-base-v2

- 🧠 High accuracy on semantic similarity benchmarks
- 🐢 Slower and heavier than MiniLMs
- ✅ Best if clustering quality is more important than speed

4. paraphrase-multilingual-MiniLM-L12-v2

- 🌐 Multilingual support (50+ languages)
- ✅ Ideal if your titles aren't only in English
- 📁 Slightly larger and slower

5. BAAI/bge-small-en or bge-base-en-v1.5

-  Newer open-source embedding models from BAAI
-  State-of-the-art performance in search & clustering tasks
-  Very competitive with OpenAI's embeddings
-  You need to set them up using HuggingFace

```
from sentence_transformers import SentenceTransformer model =  
SentenceTransformer("BAAI/bge-small-en") # or "bge-base-en-v1.5"
```

6. intfloat/e5-small-v2 or e5-base-v2

-  Powerful embeddings trained for retrieval and clustering
-  Excellent clustering behavior with dense output
-  Needs special input formatting:

```
sentences = ["query: " + title for title in titles]
```

Comparison Summary

Model Name	Speed	Accuracy	Size	Multilingual	Notes
all-MiniLM-L6-v2	 Fast	 Good	Small	 No	Best for most clustering tasks
all-MiniLM-L12-v2	 Medium	 Better	Medium	 No	Slightly better clustering
all-mpnet-base-v2	 Medium	 Best	Large	 No	High-quality clusters
paraphrase-MiniLM-L6-v2	 Fast	 Good	Small	 No	Good for semantic

Model Name	Speed	Accuracy	Size	Multilingual	Notes
paraphrase-multilingual-MiniLM-L12-v2	⚠ Medium	👍 Good	Medium	✅ Yes	similarity If you need multi-language support
bge-small-en , bge-base-en	✅ Fast → ⚠ Medium	📈 Strong	Small/Base	❌ No	Strong modern baseline
e5-small-v2 , e5-base-v2	✅ Fast → ⚠ Medium	📈 Strong	Small/Base	❌ No	Excellent for clustering with formatting

✅ Recommended Replacements Based on Your Goal (Video Title Clustering)

- For **better clustering** with similar speed → all-MiniLM-L12-v2
- For **higher accuracy** → all-mpnet-base-v2 Or bge-base-en-v1.5
- For **best multilingual support** → paraphrase-multilingual-MiniLM-L12-v2
- For **cutting-edge performance** → intfloat/e5-base-v2 Or BAAI/bge-base-en-v1.5

Let me know if you want help benchmarking a few of these on your dataset, or need clustering examples using bge or e5 .

ok, so now that i have the output like this

2014-11-08 - Warring States Japan: Sengoku Jidai - Battle

of Okehazama - Extra History - Part 1
2014-11-22 - Warring States Japan: Sengoku Jidai - The
Siege of Inabayama Castle - Extra History - Part 2
2014-12-06 - Warring States Japan: Sengoku Jidai -
Warrior Monks of Hongan-ji and Hiei - Extra History - Part
3
2014-12-20 - Warring States Japan: Sengoku Jidai - The
Death of Oda Nobunaga - Extra History - Part 4
2015-01-17 - Warring States Japan: Sengoku Jidai - How
Toyotomi Unified Japan - Extra History - Part 5
2015-01-31 - Warring States Japan: Sengoku Jidai - The
Campaign of Sekigahara - Extra History - Part 6
2015-02-14 - Warring States Japan: Sengoku Jidai - Lies -
Extra History

how do i detect the most common repeating phrase in this
cluster, which is Warring States Japan: Sengoku Jidai? this
phrase will be used as the name of the series. keep in mind
that the phrase extra history occur with just as much
frequency because it is always in the title of each video,
and i want to ignore the term extra history

Great — now you're trying to extract the **most representative series name** from a cluster
of video titles, while **ignoring generic terms** like "Extra History" .

This is essentially a **common prefix detection task**, but with some noise (like subtitles,
episode names, etc.), so we'll take a smart approach:

Goal

From a list of titles like:

Warring States Japan: Sengoku Jidai - Battle of Okehazama - Extra History - Part
...

Extract the series name:

Strategy

1. Strip boilerplate terms (like "Extra History" and "Part X") from each title.
2. Tokenize titles and compare them to find the **longest common n-gram** or prefix.
3. Optionally: prioritize sequences that occur in the **same order** across most titles.

Example Python Function

```
import re from collections import Counter def clean_title(title): # Remove
"Extra History" and "Part X" or similar suffixes title = re.sub(r'\s*-\s*Extra
History', '', title, flags=re.IGNORECASE) title = re.sub(r'\s*-\s*Part\s*\d+',
'', title, flags=re.IGNORECASE) title = title.strip() return title def
get_common_prefix(titles): cleaned_titles = [clean_title(t) for t in titles] #
Tokenize each title into words tokenized = [t.split(' - ') for t in
cleaned_titles] # Find all n-gram chunks and count frequencies ngram_counter =
Counter() for tokens in tokenized: for i in range(len(tokens)): for j in
range(i + 1, len(tokens) + 1): chunk = ' - '.join(tokens[i:j])
ngram_counter[chunk] += 1 # Only keep chunks that appear in most of the titles
min_count = max(2, len(titles) // 2) common_ngrams = [ng for ng, count in
ngram_counter.items() if count >= min_count] # Sort by length (longest first),
then by frequency common_ngrams.sort(key=lambda x: (-len(x), -
ngram_counter[x])) return common_ngrams[0] if common_ngrams else None
```

Usage Example

```
titles = [ "Warring States Japan: Sengoku Jidai - Battle of Okehazama - Extra
History - Part 1", "Warring States Japan: Sengoku Jidai - The Siege of
Inabayama Castle - Extra History - Part 2", "Warring States Japan: Sengoku
Jidai - Warrior Monks of Hongan-ji and Hiei - Extra History - Part 3",
"Warring States Japan: Sengoku Jidai - The Death of Oda Nobunaga - Extra
```

```
History - Part 4", "Warring States Japan: Sengoku Jidai - How Toyotomi Unified  
Japan - Extra History - Part 5", "Warring States Japan: Sengoku Jidai - The  
Campaign of Sekigahara - Extra History - Part 6", "Warring States Japan:  
Sengoku Jidai - Lies - Extra History", ] series_name =  
get_common_prefix(titles) print(series_name)
```



Output

```
Warring States Japan: Sengoku Jidai
```



Customization Tips

- Add other boilerplate terms to ignore (e.g., "History" , "Extra Credits").
- Adjust `min_count` threshold depending on how strict or loose you want grouping to be.

Would you like to apply this across all clusters in your dataset and auto-name them? I can help you extend the script for that too.

yes do that

Great — let's extend the script to **automatically extract series names for each HDBSCAN cluster** by:

1. Iterating through each cluster.
2. Cleaning up titles (removing generic suffixes like "Extra History", "Part X").
3. Finding the **most representative common phrase** for each cluster.
4. Assigning that phrase as the **series name**.



Final Full Script: Cluster + Auto Series Name

```

import pandas as pd import hdbscan import re from sentence_transformers import
SentenceTransformer from collections import Counter # === Step 1: Load your
CSV === df = pd.read_csv("extra_history_videos.csv", header=None) df.columns =
[ "title", "published", "views", "likes", "comments", "video_id", "duration",
"url" ] df["published"] = pd.to_datetime(df["published"]) # === Step 2: Create
title embeddings === model = SentenceTransformer("all-MiniLM-L6-v2") # or try
alternatives embeddings = model.encode(df["title"].tolist()) # === Step 3:
Cluster using HDBSCAN === clusterer = hdbscan.HDBSCAN(min_cluster_size=2,
metric="euclidean") df["series_cluster"] = clusterer.fit_predict(embeddings) #
=== Step 4: Clean titles for grouping === def clean_title(title): # Remove
common trailing patterns like "Extra History" or "Part X" title =
re.sub(r'\s*-\s*Extra History', '', title, flags=re.IGNORECASE) title =
re.sub(r'\s*-\s*Part\s*\d+', '', title, flags=re.IGNORECASE) return
title.strip() def extract_common_series_name(titles): cleaned_titles =
[clean_title(t) for t in titles] tokenized = [t.split(' - ') for t in
cleaned_titles] ngram_counter = Counter() for tokens in tokenized: for i in
range(len(tokens)): for j in range(i + 1, len(tokens) + 1): chunk = ' -
'.join(tokens[i:j]) ngram_counter[chunk] += 1 min_count = max(2, len(titles)
// 2) common_ngrams = [ng for ng, count in ngram_counter.items() if count >=
min_count] common_ngrams.sort(key=lambda x: (-len(x), -ngram_counter[x]))
return common_ngrams[0] if common_ngrams else "Unnamed Series" # === Step 5:
Assign series names per cluster === series_names = {} for cluster_id in
sorted(df["series_cluster"].unique()): if cluster_id == -1: continue # Skip
unclustered videos cluster_titles = df[df["series_cluster"] == cluster_id]
["title"].tolist() name = extract_common_series_name(cluster_titles)
series_names[cluster_id] = name # Add the series name to the dataframe
df["series_name"] = df["series_cluster"].apply(lambda cid:
series_names.get(cid, "Unclustered")) # === Step 6: Sort & Save === df =
df.sort_values(by=["series_name", "published"])
df.to_csv("extra_history_series_grouped.csv", index=False) # === Optional:
Print Summary === print("\n📖 Series Groups:") for sid, name in
series_names.items(): count = len(df[df["series_cluster"] == sid]) print(f"
[{sid}] {name} - {count} videos")

```



Output Files

- extra_history_series_grouped.csv → includes:
 - Original title

- Series cluster ID
- Auto-extracted `series_name`



Requirements

```
pip install pandas hdbscan sentence-transformers
```



Sample Output (Printed)



Series Groups:

```
[0] Warring States Japan: Sengoku Jidai — 7 videos
[1] Julius Caesar — 4 videos
[2] World War I — 5 videos
...
```

Let me know if you'd like to include unclustered videos in separate categories like "One-offs" or need a Colab-ready version of this.

where to modify the code so that each series name also shows the videos' titles?

Great — if you'd like the script to **print out each series name followed by the list of video titles** in that series, you just need to modify the " **Series Groups**" summary section at the end of the script.



Modify This Block:

Current (prints only cluster IDs and series names):

```
print("\n📖 Series Groups:") for sid, name in series_names.items(): count =
len(df[df["series_cluster"] == sid]) print(f"[{sid}] {name} - {count} videos")
```



Replace With This (to include titles):

```
print("\n📖 Series Groups with Titles:") for sid, name in
series_names.items(): cluster_df = df[df["series_cluster"] ==
sid].sort_values(by="published") print(f"\n[{sid}] {name} - {len(cluster_df)}
videos") for _, row in cluster_df.iterrows(): print(f" -
{row['published'].date()} | {row['title']}")
```



Example Output:

```
[0] Warring States Japan: Sengoku Jidai - 7 videos
- 2014-11-08 | Warring States Japan: Sengoku Jidai - Battle of Okehazama - Extr
- 2014-11-22 | Warring States Japan: Sengoku Jidai - The Siege of Inabayama Cas
- 2014-12-06 | Warring States Japan: Sengoku Jidai - Warrior Monks of Hongan-ji
...
```

This change is purely cosmetic/output-focused — it doesn't modify your CSV file or data structure. Let me know if you'd like this output **also saved to a .txt file** or formatted differently (e.g., markdown, JSON, etc.).

if i run the clustering again, will it give me different results?

Good question — and the answer is: **possibly, but not necessarily**.

It depends on a few factors, especially with **HDBSCAN** and how you're setting up the environment.

Will HDBSCAN give different results on repeated runs?

Deterministic by Default

If:

- Your input data (titles) and order are **identical**
- The **embedding model** produces **consistent output**
- No **randomness** is introduced in clustering parameters

👉 Then HDBSCAN will produce the same clusters every time. It is **deterministic**, given the same input.

When Might You Get Different Results?

1. Randomness in Embeddings

- Some models (especially when using GPUs or non-fixed seeds) may return **slightly different embeddings** for the same input each time.
- SentenceTransformers is **deterministic by default** in CPU mode — unless you've modified the backend or use stochastic inference like dropout during inference (rare).

2. Input Order Changes

- If you change the order of titles in the CSV or list, the embeddings are still the same, but HDBSCAN's clustering can change slightly in edge cases because it builds a graph internally and may resolve ties differently.

3. Parameter Changes

- Changing `min_cluster_size`, `min_samples`, or distance metric will change results.

How to Ensure Reproducibility

If reproducibility is critical:

- **Fix the input order** (e.g., sort titles before embedding).
- **Fix the model** (e.g., avoid loading different SentenceTransformer variants).
- **Use CPU** instead of GPU for sentence embedding to reduce numerical non-determinism.
- **Save embeddings to disk** after the first run if you need consistent input to HDBSCAN.
- You can also log or save the cluster labels:

```
df.to_csv("labeled_output.csv", index=False)
```

Summary

Factor	Affects Repeatability?
Title order changes	Sometimes (rare cases)
Embedding model noise	Rarely (GPU/stochastic)
Clustering parameters	Yes
Random seeds	Not applicable (HDBSCAN is deterministic)

Would you like help saving the cluster labels + embeddings for exact reuse later, or testing for consistency between two runs?

ok instead of this

```
# === Optional: Print Summary ===
print("\n📄 Series Groups with Titles:")
for sid, name in series_names.items():
    cluster_df = df[df["series_cluster"] ==
sid].sort_values(by="published")
    print(f"\n[{sid}] {name} — {len(cluster_df)} videos")
    for _, row in cluster_df.iterrows():
        print(f"  - {row['published'].date()} | {row['title']}")
```

print it to a file

Perfect — to write the **series summary (with titles)** to a file instead of printing to the console, you can replace the `print()` statements with writes to a file using Python's built-in file handling.

✅ Replace Your Print Block With This

```
# === Optional: Save Series Summary to File === output_path =
"series_summary.txt" with open(output_path, "w", encoding="utf-8") as f:
f.write("\n📄 Series Groups with Titles:\n") for sid, name in
series_names.items(): cluster_df = df[df["series_cluster"] ==
sid].sort_values(by="published") f.write(f"\n[{sid}] {name} —
{len(cluster_df)} videos\n") for _, row in cluster_df.iterrows(): f.write(f"  -
{row['published'].date()} | {row['title']}\n") print(f"✅ Series summary
written to {output_path}")
```

Result

You'll get a `series_summary.txt` file like this:

Series Groups with Titles:

```
[0] Warring States Japan: Sengoku Jidai — 7 videos
  - 2014-11-08 | Warring States Japan: Sengoku Jidai - Battle of Okehazama - Extr
  - 2014-11-22 | Warring States Japan: Sengoku Jidai - The Siege of Inabayama Cas
  ...

[1] Julius Caesar — 4 videos
  - ...
```

Let me know if you'd like a **Markdown version**, or grouped into **folders/files per series**, or even exported to **JSON for programmatic use**.

can you walk me thru why this series is unnamed? it seems like it should be named "Nazi Occultism"

```
[13] Unnamed Series — 7 videos
  - 2016-02-15 | On World War II and Nazi Symbolism: A
    Note from Extra History |
    https://www.youtube.com/watch?v=LSjFUOA2oZM
  - 2024-10-19 | Nazi Occultism 1: The Thule Society |
    European History | WW2 | Extra History |
    https://www.youtube.com/watch?v=TokYUITs6zs
  - 2024-10-26 | Nazi Occultism 2: Weird Science |
    European History | WW2 | Extra History |
    https://www.youtube.com/watch?v=6MEzUovLTMs
  - 2024-11-02 | Nazi Occultism 3: Hitler vs Magic |
    European History | WW2 | Extra History |
    https://www.youtube.com/watch?v=06fXlstwddk
```

- 2024-11-09 | Nazi Occultism 4: The Indiana Jones Stuff |
European History | WW2 | Extra History |
<https://www.youtube.com/watch?v=Tg8CpKzcFm8>
- 2024-11-16 | Nazi Occultism 5: Thor's Hammer |
European History | WW2 | Extra History |
<https://www.youtube.com/watch?v=1H-z5Dhy5c8>
- 2024-11-30 | Nazi Occultism | LIES | Extra History |
<https://www.youtube.com/watch?v=xRwnSoaNmjA>

Thanks for providing that cluster. Based on the titles, you're absolutely right: the series should be named something like **"Nazi Occultism"**.

Let's walk through **why your current code labeled it as "Unnamed Series"** — and how to fix it.

Breakdown of What Happened

Current title cleaning logic (in `clean_title`):

```
title = re.sub(r'\s*-\s*Extra History', '', title, flags=re.IGNORECASE) title  
= re.sub(r'\s*-\s*Part\s*\d+', '', title, flags=re.IGNORECASE)
```

This only removes:

- " - Extra History"
- " - Part X"

But your actual titles don't use " - Part X" consistently. Instead, some use "Nazi Occultism 1: ...", "Nazi Occultism 2: ...", and others use "Nazi Occultism | LIES" or have extra content like "European History | WW2".

So what's happening?

Why Unnamed Series Is Being Returned

The `extract_common_series_name()` function computes **n-grams** over split title chunks and looks for a common substring that:

- Appears in at least `len(titles) // 2` titles
- Is the **longest** frequent chunk

But due to:

- Different formatting (: vs |)
- Prepend/append clutter (like "European History | WW2")
- Inconsistent prefix position

...the common string "Nazi Occultism" never ends up being the **longest shared chunk** across 4+ titles in the current logic.

✅ Fix: Improve the Title Cleaning Step

✨ Step 1: Normalize separators

Replace | , : , multiple spaces etc. with a consistent delimiter.

✨ Step 2: Remove known suffixes (Extra History, LIES, etc.)

✨ Step 3: Extract leading chunks (especially before episode numbers)

✅ Improved `clean_title` Function

Replace your existing `clean_title` with this:

```
def clean_title(title): title = title.strip() # Normalize separators title =
re.sub(r'\s*|\s*', ' - ', title) # convert pipes to hyphens title =
re.sub(r'\s{2,}', ' ', title) # remove excessive spaces # Remove boilerplate
terms title = re.sub(r'-?\s*Extra History', '', title, flags=re.IGNORECASE)
title = re.sub(r'-?\s*European History', '', title, flags=re.IGNORECASE) title
= re.sub(r'-?\s*World War II', '', title, flags=re.IGNORECASE) title =
```

```
re.sub(r'-?\s*WW2', '', title, flags=re.IGNORECASE) title = re.sub(r'-?\s*LIES', '', title, flags=re.IGNORECASE) # Remove trailing episode numbers like "Nazi Occultism 4:" title = re.sub(r'(.*) (\d+\s*:\s*)', r'\1', title) # Final cleanup title = re.sub(r'\s*-\s*$', '', title) # remove trailing dashes return title.strip()
```



With this fix...

These cleaned titles:

```
Nazi Occultism
Nazi Occultism
Nazi Occultism
Nazi Occultism
Nazi Occultism
Nazi Occultism
On World War II and Nazi Symbolism: A Note from
```

...will yield "Nazi Occultism" as the **longest shared n-gram** for 6 of 7 titles, which is well over the threshold, and **will be chosen as the series name**.

The 7th one ("Note from Extra History") is thematically related, but structurally different — so it won't interfere much.



What To Do Now

1. Replace your `clean_title` function with the improved version above.
2. Rerun your script — the series should now appear as:

```
[13] Nazi Occultism — 7 videos
```

Let me know if you'd like the script to **automatically detect and remove boilerplate terms** from the *end* or *middle* of titles using a config or keyword list.

"nazi occultism" occurred on 5 videos, shouldn't that be enough?

You're absolutely right to question that — let's clarify the logic, and why 5 occurrences **should** be enough.

✅ Yes — 5 Titles Sharing "Nazi Occultism" Should Be Enough

Your current logic in `extract_common_series_name()` includes:

```
min_count = max(2, len(titles) // 2)
```

So for 7 videos, it computes:

```
min_count = max(2, 7 // 2) = max(2, 3) = 3
```

This means: any phrase that appears in at least 3 of the 7 cleaned titles should qualify.

So if "Nazi Occultism" appears in 5 of them after cleaning, it should be chosen — **unless**:

❌ One of These Issues Happens:

1. The phrase isn't tokenized consistently
 - For example, if some cleaned titles are "Nazi Occultism - Weird Science" and others "Nazi Occultism" (without dash), your current n-gram collector might treat them as different sequences.
2. The "Nazi Occultism" phrase isn't a full chunk

- Your function collects n-grams from combinations of **entire segments split by "**
 - **"** , not arbitrary spans of words.

✅ Solution: Normalize the Titles *and* Count Phrases Across Full Strings

Instead of relying only on n-grams from " - " splits, we can take a **phrase frequency approach** directly.

Here's an **improved version** of `extract_common_series_name()` that works better in these fuzzy cases:

🧠 Improved `extract_common_series_name()` Using Phrase Frequency

```
def extract_common_series_name(titles): cleaned_titles = [clean_title(t) for t
in titles] # Count all candidate leading phrases up to 5 words phrase_counter
= Counter() for title in cleaned_titles: words = title.split() for i in
range(1, min(6, len(words) + 1)): phrase = ' '.join(words[:i])
phrase_counter[phrase] += 1 # Filter candidates by frequency threshold
min_count = max(2, len(titles) // 2) frequent_phrases = [p for p in
phrase_counter if phrase_counter[p] >= min_count] if not frequent_phrases:
return "Unnamed Series" # Return the longest, most frequent phrase
frequent_phrases.sort(key=lambda p: (-len(p), -phrase_counter[p])) return
frequent_phrases[0]
```

🧪 Why This Works

- Focuses on leading words — e.g., "Nazi Occultism" will be caught even if followed by different subtitles
- Ensures consistent detection regardless of delimiter (: , | , -)

- Only returns series names that occur in a significant portion of the titles

Bottom Line

- Yes, **5 out of 7 is enough**, by your own threshold logic
- If "Nazi Occultism" is not being detected, it's due to **inconsistent title structure** or **too rigid n-gram matching**
- The updated function above should fix it and produce a correct name

Would you like me to help integrate this into your current script so you don't lose the original structure?

what is the tokenization step here?

Great question — let's break it down:

In the original function, the **tokenization step** happens here:

```
tokenized = [t.split(' - ') for t in cleaned_titles]
```

This means:

- It **splits each title** into chunks using " - " as the delimiter.
- For example:

"Nazi Occultism 2: Weird Science - European History - Extra History"

would become:

```
['Nazi Occultism 2: Weird Science', 'European History', 'Extra History']
```

Why That's a Problem

This method **does not tokenize by words**, but by **chunks separated by hyphens**.

As a result:

- The core phrase "Nazi Occultism" gets buried inside longer chunks like "Nazi Occultism 3: Hitler vs Magic".
- Unless multiple titles are **exactly identical in chunk boundaries**, you won't find "Nazi Occultism" as a shared chunk.

This limits your ability to detect **loose, word-based common phrases**, especially if there's variation in formatting (colons, numbers, subtitles, pipes, etc.).

A Better Tokenization Step

In the updated version I gave you, tokenization happens here:

```
words = title.split()
```

This splits each **entire cleaned title into individual words** — regardless of punctuation or dash usage — allowing you to build **n-gram phrases** from actual words at the beginning of the title.

For example:

```
title = "Nazi Occultism 2: Weird Science" words = ['Nazi', 'Occultism', '2:',  
'Weird', 'Science']
```

Then it constructs:

- 'Nazi'
- 'Nazi Occultism'
- 'Nazi Occultism 2:'
- 'Nazi Occultism 2: Weird'

- 'Nazi Occultism 2: Weird Science'

You can then detect that "Nazi Occultism" appears at the beginning of **multiple titles**, even if the rest of the title differs.

Summary

Tokenization Step	What It Does	Limitation
<code>title.split(' - ')</code>	Splits on " - "	Breaks when titles vary in punctuation
<code>title.split()</code> (word-based)	Tokenizes by individual words	Captures shared word phrases robustly

If you want the **best of both** approaches (structured + word-based), we can combine both tokenizers — or even use a proper NLP tokenizer (like from SpaCy or HuggingFace). Let me know how deep you want to go.

ok, so if i can cluster by the video's title, can i also cluster by the video's published date? the closer they are together the better, and outliers should be remove

Yes — you can absolutely **cluster by published date**, and you're right to think of it as a **temporal clustering problem**, where videos published close together likely belong to the same series.

HDBSCAN (or similar density-based methods) can be applied to **dates** too — treating time as a **1D or 2D numerical feature**.



Why Date-Based Clustering Makes Sense

- Extra History often publishes videos in a **weekly cadence** for a series.
- If videos are released **within a short window (e.g. a few weeks)**, they likely belong to the same series.
- Videos far apart in time are likely part of **different topics or one-offs**.



How to Cluster by Published Date



Step-by-step Strategy

1. Convert published dates to timestamps (e.g., Unix time or days since epoch).
2. Use those as input to HDBSCAN or DBSCAN.
3. Set clustering parameters so it groups videos published **close in time**.
4. Optional: combine **date + title embeddings** to guide clustering better.



Example: Date-Only Clustering with HDBSCAN

```
import pandas as pd import hdbscan import numpy as np # Load your video data
df = pd.read_csv("extra_history_videos.csv", header=None) df.columns =
["title", "published", "views", "likes", "comments", "video_id", "duration",
"url"] df["published"] = pd.to_datetime(df["published"]) # Convert date to
numeric form (e.g., days since earliest video) df["days_since_start"] =
(df["published"] - df["published"].min()).dt.days date_data =
df["days_since_start"].values.reshape(-1, 1) # Cluster using HDBSCAN on date
clusterer = hdbscan.HDBSCAN(min_cluster_size=3, min_samples=1,
metric="euclidean") df["date_cluster"] = clusterer.fit_predict(date_data) #
View results for cid in sorted(df["date_cluster"].unique()): if cid == -1:
continue # skip noise cluster = df[df["date_cluster"] ==
cid].sort_values("published") print(f"\n[Cluster {cid}] - {len(cluster)}
videos") for _, row in cluster.iterrows(): print(f" {row['published'].date()}
| {row['title']}"]
```

Handling Outliers

HDBSCAN will assign `-1` to **outlier videos** — ones published too far from others. You can either:

- Leave them as "Unclustered"
- Assign them to "One-Offs"
- Use them for manual review

Optionally Combine Title + Date

If you want to be really robust, you can cluster using a **combined feature space**:

```
from sklearn.preprocessing import MinMaxScaler import numpy as np # Scale date
and combine with title embeddings scaler = MinMaxScaler() date_scaled =
scaler.fit_transform(df["days_since_start"].values.reshape(-1, 1))
combined_features = np.hstack([embeddings, date_scaled]) # Cluster on the
combined space clusterer = hdbscan.HDBSCAN(min_cluster_size=3,
metric="euclidean") df["combined_cluster"] =
clusterer.fit_predict(combined_features)
```

This approach gives you:

- **Semantic similarity** from titles
- **Temporal proximity** from dates
→ in one clustering step.

Summary

Clustering Approach

Good For

Title-only

Topic-based grouping

Date-only

Temporal series grouping

Combined Title + Date

Most robust for episodic detection

Would you like me to help **merge your current title clustering with this date-based approach**, or experiment with date-only clustering on your data?

i want to combine BOTH title clustering AND date clustering, not replacing the former with the latter

Perfect — that's the best approach: **combine semantic similarity (from titles)** with **temporal proximity (from published dates)** to improve your clustering.

This gives you series that are:

- **Topically similar** (based on the title)
- **Released close together** (based on date)

Let's walk through how to do this effectively.

How to Combine Title + Date for Clustering

Step 1: Get embeddings from titles

Using `sentence-transformers`, as you already do:

```
from sentence_transformers import SentenceTransformer model =  
SentenceTransformer("all-MiniLM-L6-v2") title_embeddings =  
model.encode(df["title"].tolist()) # shape: (N, D)
```

Step 2: Convert dates to numerical form

```
from sklearn.preprocessing import MinMaxScaler import numpy as np
df["published"] = pd.to_datetime(df["published"]) df["days_since_start"] =
(df["published"] - df["published"].min()).dt.days date_numeric =
df["days_since_start"].values.reshape(-1, 1) # Scale dates to [0, 1] to match
title embedding scale date_scaled = MinMaxScaler().fit_transform(date_numeric)
```

Step 3: Concatenate title and date features

```
combined_features = np.hstack([title_embeddings, date_scaled]) # shape: (N,
D+1)
```

This gives HDBSCAN both kinds of information:

- The **main content signal** from the title
- The **temporal context** as a lightweight influence

Step 4: Cluster with HDBSCAN

```
import hdbscan clusterer = hdbscan.HDBSCAN(min_cluster_size=3, min_samples=1,
metric="euclidean") df["combined_cluster"] =
clusterer.fit_predict(combined_features)
```

Benefits of This Approach

Feature	Why it Helps
<code>title_embeddings</code>	Detects semantic similarity (topic match)
<code>date_scaled</code>	Enforces time-local grouping (weekly uploads)

Feature

Why it Helps

Combined

Groups videos in the same topic + time window

This avoids common pitfalls like:

- Different series with similar titles (e.g., "World War I" vs "World War II")
- The same topic recurring years later

Optional: Weighing One Feature More

You can **scale** the date more (or less) depending on how important time proximity is to you:

```
# Give time more or less influence weight = 0.5 # try 0.2 to 1.0 depending on
how strong you want time to influence date_scaled = date_scaled * weight
combined_features = np.hstack([title_embeddings, date_scaled])
```

Try adjusting `weight` to tune how strict the temporal alignment should be.

Would you like a full plug-and-play version of this integrated into your current script (with series naming and summary output)?

i want the title clustering to run first. then for each cluster, the date clustering algorithm should run, and remove any entry that doesn't match. in effect, the date clustering algorithm is further pruning the result of the title clustering step

Got it — that's a smart two-stage approach:



Your Desired Flow:

1. Run **title-based clustering first** (as you do now with sentence embeddings + HDBSCAN).
2. For **each resulting cluster** from that step:
 - Run a **secondary clustering step on dates only**.
 - Use this to **remove outliers** that are too far away in time (temporal outliers).
3. The final result is:
 - **Semantically consistent** clusters (by topic)
 - **Temporally coherent** (videos are close together in publish date)



Why This Works Well

This lets you:

- Prioritize **topic similarity** first (most important)
- Then **filter out anomalies** using time proximity (e.g. reboots, one-offs, long gaps)



How to Implement This



Step-by-step Implementation

1. Run title-based clustering (you already have this):

```
# Step 1: Generate embeddings from titles from sentence_transformers import
SentenceTransformer model = SentenceTransformer("all-MiniLM-L6-v2")
title_embeddings = model.encode(df["title"].tolist()) # Step 2: Run HDBSCAN on
title_embeddings import hdbscan title_clusterer =
hdbscan.HDBSCAN(min_cluster_size=3, min_samples=1, metric="euclidean")
df["title_cluster"] = title_clusterer.fit_predict(title_embeddings)
```

2. For each title-based cluster, run a date-only cluster to prune temporal outliers

```
from sklearn.preprocessing import MinMaxScaler import numpy as np # Convert
published date to numeric df["published"] = pd.to_datetime(df["published"])
df["days_since_start"] = (df["published"] - df["published"].min()).dt.days #
Prepare new column for pruned clusters df["final_cluster"] = -1 cluster_id = 0
# For each title-based cluster for t_cluster in df["title_cluster"].unique():
if t_cluster == -1: continue # skip unclustered titles group =
df[df["title_cluster"] == t_cluster].copy() # Run HDBSCAN on days since start
(1D) dates = group["days_since_start"].values.reshape(-1, 1) scaled_dates =
MinMaxScaler().fit_transform(dates) # Run HDBSCAN on scaled dates to remove
time-based outliers date_clusterer = hdbscan.HDBSCAN(min_cluster_size=3,
min_samples=1, metric="euclidean") group["date_subcluster"] =
date_clusterer.fit_predict(scaled_dates) # Keep only the largest date-based
subcluster for subcluster_id in group["date_subcluster"].unique(): if
subcluster_id == -1: continue # remove date outliers sub =
group[group["date_subcluster"] == subcluster_id] if len(sub) < 3: continue #
discard very small subclusters # Assign final cluster ID df.loc[sub.index,
"final_cluster"] = cluster_id cluster_id += 1
```

Final Output Columns

Column	Description
title_cluster	Initial cluster based on title embeddings
date_subcluster	Date-based refinement inside each cluster
final_cluster	Combined result: topic match + time match

Result: Cleaner, More Accurate Series Grouping

- You can now build `series_name` based on `final_cluster`
- Any video that doesn't match **topic or time** is dropped (`final_cluster = -1`)

Would you like me to plug this into your full pipeline (with cleaned titles, auto series name, and summary export)?

yes. hreres my curret code. try to have the ouput that i currently have

```
import pandas as pd
import hdbscan
import re
from sentence_transformers import SentenceTransformer
from collections import Counter

# === Step 1: Load your CSV ===
df = pd.read_csv("all extra history.csv", header=None)
df.columns = [
    "title", "published", "views", "likes", "comments",
    "video_id", "duration", "url"
]
df["published"] = pd.to_datetime(df["published"])

# === Step 2: Create title embeddings ===
model = SentenceTransformer("all-MiniLM-L6-v2") # or
try alternatives
embeddings = model.encode(df["title"].tolist())

# === Step 3: Cluster using HDBSCAN ===
clusterer = hdbscan.HDBSCAN(min_cluster_size=2,
metric="euclidean")
df["series_cluster"] = clusterer.fit_predict(embeddings)

# === Step 4: Clean titles for grouping ===
def clean_title(title):
    # Remove common trailing patterns like "Extra History"
    or "Part X"
    title = re.sub(r'\s*-\s*Extra History', "", title,
```

```

flags=re.IGNORECASE)
    title = re.sub(r'\s*-\s*Part\s*\d+', '', title,
flags=re.IGNORECASE)
    return title.strip()

def extract_common_series_name(titles):
    cleaned_titles = [clean_title(t) for t in titles]
    tokenized = [t.split(' - ') for t in cleaned_titles]
    print(tokenized)

    ngram_counter = Counter()
    for tokens in tokenized:
        for i in range(len(tokens)):
            for j in range(i + 1, len(tokens) + 1):
                chunk = ' - '.join(tokens[i:j])
                ngram_counter[chunk] += 1

    min_count = max(2, len(titles) // 2)
    common_ngrams = [ng for ng, count in
ngram_counter.items() if count >= min_count]
    common_ngrams.sort(key=lambda x: (-len(x), -
ngram_counter[x]))

    return common_ngrams[0] if common_ngrams else
"Unnamed Series"

```

```

# === Step 5: Assign series names per cluster ===
series_names = {}
for cluster_id in sorted(df["series_cluster"].unique()):
    if cluster_id == -1:
        continue # Skip unclustered videos
    cluster_titles = df[df["series_cluster"] == cluster_id]
["title"].tolist()
    name = extract_common_series_name(cluster_titles)
    series_names[cluster_id] = name

# Add the series name to the dataframe
df["series_name"] = df["series_cluster"].apply(lambda cid:
series_names.get(cid, "Unclustered"))

```

```

# === Step 6: Sort & Save ===
df = df.sort_values(by=["series_name", "published"])
df.to_csv("extra_history_series_grouped.csv", index=False)

# === Optional: Save Series Summary to File ===
output_path = "series_summary.txt"

with open(output_path, "w", encoding="utf-8") as f:
    f.write("📖 Series Groups with Titles:\n")
    for sid, name in series_names.items():
        cluster_df = df[df["series_cluster"] ==
sid].sort_values(by="published")
        f.write(f"\n[{sid}] {name} — {len(cluster_df)} videos\n")
        for _, row in cluster_df.iterrows():
            # f.write(f" - {row['published'].date()} | {row['title']} |
{row['url']}\n")
            f.write(f" ~ {row['title']}\n")

print(f"✅ Series summary written to {output_path}")

```

I've prepared the revised version of your script that first clusters by title, then prunes each cluster based on publish date proximity. To edit or run it, you'll need to log in to ChatGPT and open the canvas tool. Let me know if you want the complete script pasted here instead.

put the script here

Absolutely — here's the full revised version of your script that:

1. Clusters videos by **title** using sentence embeddings + HDBSCAN.
2. For each title-based cluster, performs a **secondary HDBSCAN clustering on publish dates** to remove temporal outliers.

3. Assigns series names and exports both the cleaned CSV and a summary text file with titles grouped under each final cluster.

✓ Updated Script

```
import pandas as pd
import hdbscan
import re
from sentence_transformers import SentenceTransformer
from collections import Counter
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# === Step 1: Load CSV ===
df = pd.read_csv("all_extra_history.csv", header=None)
df.columns = [ "title", "published", "views", "likes", "comments", "video_id", "duration", "url" ]
df["published"] = pd.to_datetime(df["published"])
df["days_since_start"] = (df["published"] - df["published"].min()).dt.days

# === Step 2: Create title embeddings ===
model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = model.encode(df["title"].tolist())

# === Step 3: Initial clustering by title ===
title_clusterer = hdbscan.HDBSCAN(min_cluster_size=2, metric="euclidean")
df["title_cluster"] = title_clusterer.fit_predict(embeddings)

# === Step 4: Clean titles ===
def clean_title(title):
    title = title.strip()
    title = re.sub(r'\s*\|\s*', ' - ', title) # normalize pipes
    title = re.sub(r'\s*-\s*Extra History', '', title, flags=re.IGNORECASE)
    title = re.sub(r'\s*-\s*European History', '', title, flags=re.IGNORECASE)
    title = re.sub(r'\s*-\s*World War II', '', title, flags=re.IGNORECASE)
    title = re.sub(r'\s*-\s*WW2', '', title, flags=re.IGNORECASE)
    title = re.sub(r'\s*-\s*LIES', '', title, flags=re.IGNORECASE)
    title = re.sub(r'\s*-\s*Part\s*\d+', '', title, flags=re.IGNORECASE)
    title = re.sub(r'(.*)\(\d+\s*:\s*\s*)', r'\1', title) # remove episode numbers
    return title.strip()

def extract_common_series_name(titles):
    cleaned_titles = [clean_title(t) for t in titles]
    tokenized = [t.split(' - ') for t in cleaned_titles]
    ngram_counter = Counter()
    for tokens in tokenized:
        for i in range(len(tokens)):
            for j in range(i + 1, len(tokens) + 1):
                chunk = ' - '.join(tokens[i:j])
                ngram_counter[chunk] += 1
    min_count = max(2, len(titles) // 2)
    common_ngrams = [ng for ng, count in ngram_counter.items() if count >= min_count]
    common_ngrams.sort(key=lambda x: (-len(x), -ngram_counter[x]))
    return common_ngrams[0] if common_ngrams else "Unnamed Series"

# === Step 5: Date-based pruning within title clusters ===
df["final_cluster"] = -1
final_series_names = {}
final_cluster_id = 0
for t_cluster in df["title_cluster"].unique():
    if t_cluster == -1:
        continue
    group = df[df["title_cluster"] == t_cluster].copy()
    # Run HDBSCAN on date only (scaled)
    date_values = group["days_since_start"].values.reshape(-1, 1)
    date_scaled = MinMaxScaler().fit_transform(date_values)
    date_clusterer = hdbscan.HDBSCAN(min_cluster_size=2, min_samples=1, metric="euclidean")
    group["date_subcluster"] = date_clusterer.fit_predict(date_scaled)
    for sub_id in group["date_subcluster"].unique():
        if sub_id == -1:
            continue
        sub =
```

```

group[group["date_subcluster"] == sub_id] if len(sub) < 2: continue # Assign
final_cluster ID and extract series name df.loc[sub.index, "final_cluster"] =
final_cluster_id name = extract_common_series_name(sub["title"].tolist())
final_series_names[final_cluster_id] = name final_cluster_id += 1 # Add series
name to dataframe df["series_name"] = df["final_cluster"].apply(lambda cid:
final_series_names.get(cid, "Unclustered")) # === Step 6: Sort & Save === df =
df.sort_values(by=["series_name", "published"])
df.to_csv("extra_history_series_grouped.csv", index=False) # === Step 7: Save
Summary to File === output_path = "series_summary.txt" with open(output_path,
"w", encoding="utf-8") as f: f.write("📁 Series Groups with Titles:\n") for
sid, name in final_series_names.items(): cluster_df = df[df["final_cluster"]
== sid].sort_values(by="published") f.write(f"\n[{sid}] {name} -
{len(cluster_df)} videos\n") for _, row in cluster_df.iterrows(): f.write(f" ~
{row['title']}\n") print(f"✅ Series summary written to {output_path}")

```

Notes

- You can adjust `min_cluster_size` for both title and date HDBSCAN steps to be stricter or looser.
- Temporal outliers in each series (e.g., years apart) will be excluded automatically.
- If you want to include video URLs or dates in the summary file, let me know — it's a quick tweak.

Would you like this also exported as Markdown or JSON for a cleaner UI or further processing?

i got this result, which i dont really want. should this be one instead of two clusters?

[10] Italian History — 4 videos

- 2024-12-28 | The Pazzi Conspiracy 3: Street Justice | Italian History | Extra History |

<https://www.youtube.com/watch?v=EGmv70-WQyo>

- 2025-01-04 | The Pazzi Conspiracy 4: Well Executed |

Italian History | Extra History |

<https://www.youtube.com/watch?v=-dDwWxU8BiA>

- 2025-01-11 | The Bonfire of the Vanities | Italian History | Extra History | https://www.youtube.com/watch?v=eEiKk-us_IA

- 2025-01-15 | The Pazzi Conspiracy | LIES | Extra History | <https://www.youtube.com/watch?v=pdkWyqjD1M0>

[11] Italian History — 2 videos

- 2024-12-14 | The Pazzi Conspiracy 1: The Pope's Plot | Italian History | Extra History |

<https://www.youtube.com/watch?v=3a80aHHlqZ0>

- 2024-12-21 | The Pazzi Conspiracy 2: Blood in the Cathedral | Italian History | Extra History | <https://www.youtube.com/watch?v=iZ7ZwxcfMmM>

Yes — this **should ideally be a single cluster**, and your current result suggests that the **temporal pruning step (date clustering)** is too aggressive. Here's why it split into two:

What's Happening

1. **Title clustering** grouped all 6 videos together correctly (they're all about the *Pazzi Conspiracy / Italian History*).
2. Then, within that title cluster, we ran **HDBSCAN again just on publish dates** to prune temporal outliers.
3. Because HDBSCAN is a density-based algorithm:
 - If there's a **gap in the upload schedule** (e.g. more than ~2 weeks),
 - It can split the group into two **subclusters** — even though they belong together topically and narratively.

In your case, a 2–3 week gap between videos caused this unwanted split.



How to Fix This

Option 1: Loosen the time-based HDBSCAN pruning

In the `HDBSCAN(min_cluster_size=2, ...)` call inside the temporal step, increase `min_cluster_size` and/or `min_samples` to make the algorithm more tolerant:

```
# Loosen date pruning
date_clusterer = hdbscan.HDBSCAN(min_cluster_size=2,
min_samples=1, metric="euclidean")
```

→ Try changing it to:

```
date_clusterer = hdbscan.HDBSCAN(min_cluster_size=2, min_samples=2,
metric="euclidean")
```

Or even:

```
date_clusterer = hdbscan.HDBSCAN(min_cluster_size=2, min_samples=3,
metric="euclidean")
```

This makes it less sensitive to small gaps.

Option 2: Use DBSCAN with a fixed max gap in days

A cleaner and more deterministic alternative is to **replace HDBSCAN with DBSCAN** for the temporal step and set a clear `eps` threshold in **days**, like `eps=20`:

```
from sklearn.cluster import DBSCAN # Instead of HDBSCAN, use DBSCAN for
tighter control
db = DBSCAN(eps=0.07, min_samples=2) # 0.07 = ~20 days if
scaled from 0-1
group["date_subcluster"] = db.fit_predict(date_scaled)
```

This lets you explicitly control how far apart episodes can be and still be in the same cluster.



Recommendation

Use DBSCAN with a time gap limit if:

- You want clear control over how far apart episodes can be.

Stick with **HDBSCAN** (and loosen it) if:

- You're OK with a little flexibility and want automatic handling.



Optional: Merge small date-subclusters back into main group

If you want to keep using HDBSCAN but **don't want it to split highly coherent series**, another idea is:

- Keep the largest temporal subcluster
- Merge any smaller subclusters that are within X days from it

I can help you implement that logic too if you'd like.

Would you like me to modify your script now to use DBSCAN for the temporal filtering with a `max_days_between_episodes` parameter?

help me understand more. the dates are dec 14, dec 21 for one cluster, THEN dec 28, jan 4, jan 11, jan 15. there's 7 days between dec 21 and dec 28, just like between all videos, so how can there be more than 2 week gap?

Excellent observation — and you're 100% right to question it. Let's break this down precisely:



Your Video Dates

Title	Date
The Pazzi Conspiracy 1	2024-12-14
The Pazzi Conspiracy 2	2024-12-21
The Pazzi Conspiracy 3	2024-12-28
The Pazzi Conspiracy 4	2025-01-04
The Bonfire of the Vanities	2025-01-11
The Pazzi Conspiracy (LIES)	2025-01-15

There is exactly a **7-day gap** between each episode, then a **4-day gap** for the final one.

So Why Did HDBSCAN Split It?

Because HDBSCAN doesn't cluster based on exact gaps — it clusters by **density**, not distance. Here's how that plays out:

1. HDBSCAN clusters by relative density, not distance

- It looks at how many neighbors are nearby within a local region.
- If the cluster is sparse (even if evenly spaced), it may not form a dense enough region.
- So even if the gaps are regular (7 days), the **spread** of the entire time cluster (Dec 14 → Jan 15 = 32 days) may make it too “thin” to count as a single dense region.

2. Date values are scaled, so 7-day gaps become small numbers

Let's say we scale the days from 0 to 1 (via `MinMaxScaler`):

- 2024-12-14 might become 0.00
- 2025-01-15 might become 1.00
- Each 7-day step is now ~0.2 apart
- That's relatively **far apart** in scaled space for a small cluster — especially if `min_cluster_size=3`

If one end of the cluster (e.g., the first two videos) isn't connected to enough points, **HDBSCAN** drops it into a separate cluster, even though semantically it's part of the same group.

What You Can Do

Solution: Use DBSCAN with a fixed `eps` value

This fixes your issue because:

- You set the **maximum allowed gap (in days)** manually.
- If you say “any episodes published ≤ 10 days apart go together,” DBSCAN obeys that literally.

```
from sklearn.cluster import DBSCAN # Use raw days, not scaled dates =
group["days_since_start"].values.reshape(-1, 1) # Set DBSCAN eps as the max
gap in days — e.g., 10 days dbscan = DBSCAN(eps=10, min_samples=2,
metric='euclidean') group["date_subcluster"] = dbscan.fit_predict(dates)
```

This will correctly keep your 6 Pazzi Conspiracy videos together — as their **max distance** between any two adjacent episodes is just 7 days, well under 10.

Summary

Method	Behavior	Why You Saw the Split
HDBSCAN	Clusters by density	Evenly spaced 7-day videos can still look “sparse”
DBSCAN	Clusters by distance limit	You define “close enough” (e.g., ≤ 10 days)

Would you like me to modify your script to switch the date-based pruning from **HDBSCAN** to **DBSCAN** with a 10-day window? It'll fix this case and give you more reliable behavior.

yes, try dbscan

Great — here's your updated script using **DBSCAN** for the date-based pruning step. This ensures episodes released within a **10-day window** remain in the same cluster, solving the “Pazzi Conspiracy” split issue.

✓ Updated Script with DBSCAN for Temporal Clustering

```
import pandas as pd import hdbscan import re from sentence_transformers import
SentenceTransformer from collections import Counter from sklearn.preprocessing
import MinMaxScaler from sklearn.cluster import DBSCAN import numpy as np #
=== Step 1: Load CSV === df = pd.read_csv("all extra history.csv",
header=None) df.columns = [ "title", "published", "views", "likes",
"comments", "video_id", "duration", "url" ] df["published"] =
pd.to_datetime(df["published"]) df["days_since_start"] = (df["published"] -
df["published"].min()).dt.days # === Step 2: Create title embeddings === model
= SentenceTransformer("all-MiniLM-L6-v2") embeddings =
model.encode(df["title"].tolist()) # === Step 3: Initial clustering by title
=== title_clusterer = hdbscan.HDBSCAN(min_cluster_size=2, metric="euclidean")
df["title_cluster"] = title_clusterer.fit_predict(embeddings) # === Step 4:
Clean titles === def clean_title(title): title = title.strip() title =
re.sub(r'\s*\|\s*', ' - ', title) title = re.sub(r'\s*-\s*Extra History', '',
title, flags=re.IGNORECASE) title = re.sub(r'\s*-\s*European History', '',
title, flags=re.IGNORECASE) title = re.sub(r'\s*-\s*World War II', '', title,
flags=re.IGNORECASE) title = re.sub(r'\s*-\s*WW2', '', title,
flags=re.IGNORECASE) title = re.sub(r'\s*-\s*LIES', '', title,
flags=re.IGNORECASE) title = re.sub(r'\s*-\s*Part\s*\d+', '', title,
flags=re.IGNORECASE) title = re.sub(r'(\..*?)\(\d+\s*:\s*)', r'\1', title) return
title.strip() def extract_common_series_name(titles): cleaned_titles =
[clean_title(t) for t in titles] tokenized = [t.split(' - ') for t in
cleaned_titles] ngram_counter = Counter() for tokens in tokenized: for i in
range(len(tokens)): for j in range(i + 1, len(tokens) + 1): chunk = ' -
'.join(tokens[i:j]) ngram_counter[chunk] += 1 min_count = max(2, len(titles))
```

```
// 2) common_ngrams = [ng for ng, count in ngram_counter.items() if count >=
min_count] common_ngrams.sort(key=lambda x: (-len(x), -ngram_counter[x]))
return common_ngrams[0] if common_ngrams else "Unnamed Series" # === Step 5:
Date-based pruning within title clusters using DBSCAN === df["final_cluster"]
= -1 final_series_names = {} final_cluster_id = 0 # Max allowed days between
episodes in a series max_gap_days = 10 for t_cluster in
df["title_cluster"].unique(): if t_cluster == -1: continue group =
df[df["title_cluster"] == t_cluster].copy() # Use raw days for DBSCAN (no
scaling) dates = group["days_since_start"].values.reshape(-1, 1) dbscan =
DBSCAN(eps=max_gap_days, min_samples=2, metric='euclidean')
group["date_subcluster"] = dbscan.fit_predict(dates) for sub_id in
group["date_subcluster"].unique(): if sub_id == -1: continue sub =
group[group["date_subcluster"] == sub_id] if len(sub) < 2: continue # Assign
final cluster ID and series name df.loc[sub.index, "final_cluster"] =
final_cluster_id name = extract_common_series_name(sub["title"].tolist())
final_series_names[final_cluster_id] = name final_cluster_id += 1 # Add series
name to dataframe df["series_name"] = df["final_cluster"].apply(lambda cid:
final_series_names.get(cid, "Unclustered")) # === Step 6: Sort & Save === df =
df.sort_values(by=["series_name", "published"])
df.to_csv("extra_history_series_grouped.csv", index=False) # === Step 7: Save
Summary to File === output_path = "series_summary.txt" with open(output_path,
"w", encoding="utf-8") as f: f.write("📖 Series Groups with Titles:\n") for
sid, name in final_series_names.items(): cluster_df = df[df["final_cluster"]
== sid].sort_values(by="published") f.write(f"\n[{sid}] {name} -
{len(cluster_df)} videos\n") for _, row in cluster_df.iterrows(): f.write(f" -
{row['published'].date()} | {row['title']} | {row['url']}\n") print(f"✅
Series summary written to {output_path}")
```

Try It With:

- max_gap_days = 10 (default)
- Try 7 or 14 if you want stricter/looser grouping

Let me know if you'd like to:

- Include unclustered videos in a separate group
- Export each series as its own file

- Visualize the clusters by timeline or topic

Want to test this specific "Pazzi Conspiracy" case to confirm it now keeps all six videos together?